

拾日软件设计资料

版本：1.0.0+8。本文根据当前项目代码编写，介绍软件要解决的问题、主要功能、程序结构和数据处理方式。

一、软件要解决什么问题

大学生的时间安排通常分散在教务课表、群通知和自己的待办里。课表相对固定，通知可能只给开始时间，任务又常常只有截止日期。单独记录这些内容并不难，难的是临时变化后判断哪些安排受影响，以及任务还能放在什么时间做。

拾日将这些信息放在同一个学期内管理。用户可以查看课表，记录活动和任务，让助手整理通知，再为任务安排学习时间。软件生成的内容可以核对、修改和确认；不能确定的时间会保留未知。

二、面向谁使用

主要使用者是需要管理课程、作业、考试和课外活动的大学生。典型使用过程如下：

1. 创建学期，填写开学日期和节次时间，导入或手工录入课表。
2. 在今日页查看当前和后续安排，在日程页查看周课表。
3. 将临时通知交给助手整理，核对时间、地点和冲突后保存。
4. 在任务页记录待办，填写剩余耗时，设置自己允许学习的时段。
5. 生成学习方案，核对后保存。临时活动影响方案时，再重新安排。

目前交付的是Android App。后端同时负责账户、数据保存、助手处理和排程，App不是完全离线运行的软件。

三、主要功能

1. 今日与日程

今日页集中展示当天和接下来要关注的内容。日程页提供周课表与列表，课程、临时活动和已安排的学习时间都可以在各自的详情中查看。

课程按学期、星期、周次和节次保存，再展开为具体日期上的课次。这样才能只调整某一次课，而不影响整门课程在其他周的安排。

2. 任务与学习安排

任务记录“要完成什么”，例如实验报告。学习安排记录“准备什么时候做”，例如周六19:00—19:30写报告。排了30分钟不等于已经做完30分钟，任务进度需要另外更新。

任务可以暂时没有截止日期或耗时估计。记录时不必补齐所有信息；自动排程时才需要足够的信息来判断能否安排。普通任务没有填写开始时间，不会一律被认定为不能开始；明确要等待审批或材料的任务则保留等待条件。

3. 通知助手

用户可以输入文字，也可以提供图片或语音。图片和语音先在服务器上转成文字，文字模型再提取事项并调用项目提供的查询和修改工具。

助手可以查日程、找空档、整理通知、提出修改或学习方案。需要用户决定的操作会先展示内容供核对。模型不能直接绕过服务端检查修改数据库。

4. 提醒

App将符合条件的提醒交给Android本地通知系统，并在事项变更后更新提醒。权限和系统电量管理会影响实际到达；“退到后台”和系统设置里的“强制停止”不是同一种情况。

四、整体结构

系统由Flutter客户端、FastAPI后端、数据库和后台处理进程组成。

客户端负责页面、输入、显示结果、登录状态和本地提醒。后端负责验证请求、查找当前用户的数据、检查冲突、生成方案并保存修改。PostgreSQL用于正式数据，测试可以使用临时SQLite。

后台进程处理耗时较长的模型请求和媒体识别。API收到请求后先保存任务，App再查询处理进度，避免一次接口调用长时间占住连接。

部分	主要技术	项目位置
Android客户端	Flutter、Riverpod、go_router	apps/mobile/lib
后端接口	Python、FastAPI	services/api/app
数据访问与升级	SQLAlchemy、Alembic、PostgreSQL	models.py、database.py、alembic
文字助手	DeepSeek工具调用、LangGraph	agent_model.py、agent_runtime.py、agent_tools.py
学习排程	OR-Tools CP-SAT	scheduler.py、replanner.py
图片与语音识别	RapidOCR、SenseVoice / sherpa-onnx	media_recognition.py、media_worker.py
本地提醒	flutter_local_notifications	features/items/android_notifications.dart

五、客户端设计

客户端主入口是main.dart，应用初始化和路由在semester_app.dart。页面按功能放在features目录，网络请求和本地缓存放在core目录，共用控件和主题放在ui目录。

底部四个入口分别是今日、日程、任务和学期。助手作为共用入口，用户可以从当前页面发起问题。页面上下文只帮助理解正在浏览什么，不能把用户说的“明天”自动改成正在浏览周的某一天。

网络会话保存在系统安全存储中。普通缓存用于恢复界面和已获取内容，服务端仍是正式保存结果的来源。切换账号或请求失效时，要避免旧请求把内容显示到新账号下。

共用组件负责按钮、表单、时间选择和弹层，减少不同页面在间距和操作方式上的差异。动画应说明页面或内容如何变化，并支持系统减少动画设置。设计细节见界面文档。

六、数据怎么组织

用户拥有自己的学期。课程、任务、活动、学习偏好和学习方案都关联用户与学期，查询不能只凭一个对象编号就返回数据。

数据	作用
学期	开学日期、周数和节次时间
课程	星期、适用周次、节次、地点等重复上课信息
活动	会议、彩排等按日期记录的安排
任务	待完成内容、截止信息、剩余耗时和开始条件
学习时段	每周允许学习的时间，以及临时不能使用的时间
学习方案与时间块	尚未确认的方案，以及确认后具体执行时间
修改记录与操作结果	保存前后的内容，用于查询历史、避免重复保存和撤销
对话与处理任务	助手上下文、处理状态、待确认内容和结果

同一条记录有自己的版本号，整个学期另有修改编号。用户看过方案后，如果相关数据又被修改，保存端会重新检查，防止旧方案覆盖新内容。

七、几个关键流程

1. 导入课程

用户在App内打开已适配的教务页面并自行登录。页面读取脚本提取课表，App展示待导入内容。用户确认学期和课程后才写入，重复导入也要检查是否会替换现有内容。

学校页面会变化，所以读取失败、表格不完整或学期不匹配时，应让用户核对或改用手工录入，不能把空结果当成导入成功。

2. 记录只有开始时间的活动

假设课程A是09:00—10:00，课程B是10:10—11:00。通知只说09:15开会，没有散会时间。

软件可以确定会议开始点与课程A冲突，但不能据此确定课程B也一定无法参加。用户可以补充时间、明确保留重叠，或者记录已经办妥的具体课次请假。没有结束时间仍可以记录活动，不应要求请全天假。

请假记录只影响选中的课次。默认日程界面隐藏该次课，原课程和其他周次继续保留。

3. 自动安排学习时间

软件先取用户允许学习的时间，再扣除课程、固定活动、临时禁用时间和已有学习安排。随后根据任务剩余耗时、开始条件、截止时间和是否可拆分，生成候选方案。

完整安排放不下时，软件应说明原因。用户明确允许部分安排后，才可以保留一部分，并告诉用户还有多少分钟没有排进去。生成方案不会自动把任务标为完成。

4. 临时变化后的重新安排

新的会议占用了原来的学习时间时，先让用户核对冲突并确认保存会议，再由用户决定是否调整个人计划。重新安排需要另外核对和确认；计算时保留工作量和已固定的时间块，优先减少被移动的任务数，再减少移动距离。

这是“尽量少打乱原计划”的设计，不是简单把所有任务重新向前塞。求解器负责搜索方案，项目负责定义这些规则和目标，并在保存前再次检查结果。

5. 保存与撤销

用户确认后，后端检查账户、记录版本、当前时间和冲突，成功才一起保存相关修改。重复提交同一操作时返回已有结果，不重复添加。

撤销会恢复这次操作修改的内容，但不能破坏后续已经发生的安排。例如新会议仍占着原来的学习时间，就不能通过撤销重排把学习任务放回冲突位置。

八、安全与异常处理

登录密码使用Argon2保存哈希，不保存明文密码。访问凭据保存在客户端安全存储中，后端保存其哈希，并设置有效期。

教务登录由用户在网页中完成，不应把教务密码放入普通文档、演示文件或日志。模型密钥只配置在后端。上传图片和录音应与用户关联，删除学期时也要处理对应文件。

网络中断、模型超时、识别失败和数据已修改都可能发生。软件需要展示原因并保留用户输入；未保存成功时，界面不能先把任务划掉或显示为已完成。

九、测试与目前限制

项目有后端测试、Flutter测试和学校页面读取测试。本次另在Android 13模拟器和隔离后端上检查了通知整理、指定课次请假、方案确认、修改撤销和重新安排。详细结果见测试报告和原始文件。

测试数据是合成样本，不能当作实际学生试用。当前材料也不证明所有学校都能导入、所有手机后台都能提醒，或AI能正确处理任意通知。真实设备、更多通知样本和实际使用体验仍需继续测试。

十、继续维护时怎么做

新增页面时先复用现有控件，新增业务规则时先找对应后端模块和测试。涉及时间、冲突、保存或撤销的修改，应同时检查正常操作和失败情况。

版本、依赖和运行命令分别以pubspec.yaml、requirements.lock、Docker配置和scripts目录为准。本资料说明当前设计；代码变化后，应同步更新相关段落，而不是不断追加阶段记录。